

SpotLight: Title is TBD

Anonymous Author(s)

Submission Id: #xxx₁

Abstract

CCS Concepts

• Computer systems organization → Cloud computing; • Software and its engineering → Scheduling; Software performance; • Computing methodologies → Planning and scheduling.

Keywords

xxx, yyy, zzz

1 Introduction

Hybrid spot/on-demand strategies [12, 37] have been extensively studied.

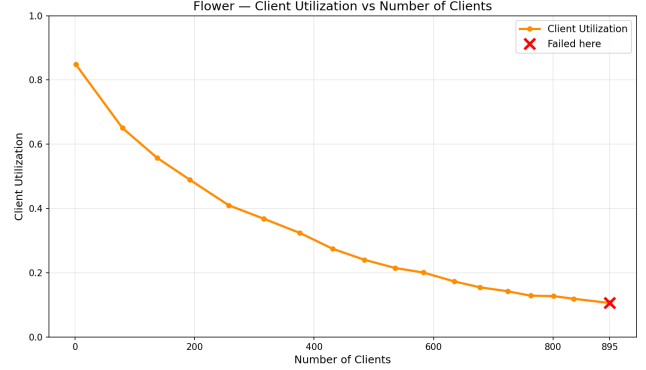
2 Motivation

Federated learning (FL) is now widely used for training machine learning models without centralizing training data [21]. As client populations grow, the infrastructure responsible for aggregating model updates must scale accordingly. Production FL deployments already coordinate tens of millions of devices, with system designers targeting deployments at the scale of billions of devices [3], placing significant demand on the aggregation layer. Existing frameworks such as Flower [2], IBM FL [20], FATE [19], and Papaya [9] are designed as single aggregator systems. In production FL deployments, the aggregation layer is responsible for managing the lifecycle of connecting clients, selecting and forwarding devices into training rounds, and combining their model updates into a global model. This breadth of responsibility places significant resource demands on a single aggregator, and as the number of clients grows, single aggregators become bottlenecks that increase the wall clock time of training, as clients are forced to wait longer for the aggregator to complete a round before receiving the next set of updates [11].

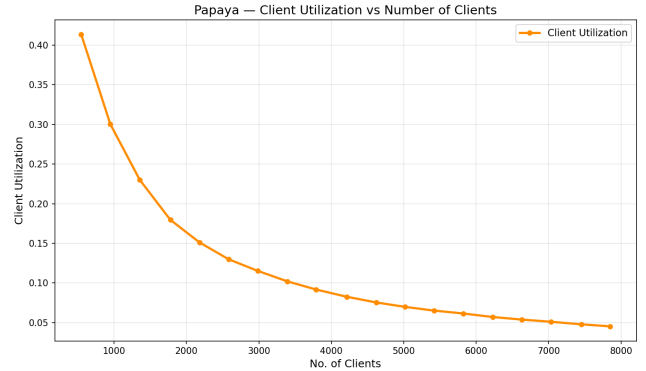
Papaya [9] and Flower [2] are both single-aggregator FL systems, though their execution models differ fundamentally. FL payload sizes vary widely, from 22MB for a model with 11 million parameters [6] to over 140GB for large language models [23], and in a single-aggregator system every update must be received, validated, and processed by that one node. As client load grows, this concentration of responsibility makes the aggregator a bottleneck. Flower’s synchronous design exposes this limitation at modest scale, causing training to stall Figure 1a. Papaya’s asynchronous execution model raises this threshold considerably, but a similar failure mode emerges at higher client load Figure 1b.

A straightforward response to the limitations of a single aggregator is to add aggregators as client load increases. The stateful nature of FL aggregation, however, makes this non-trivial. Horizontally scaling the aggregation layer introduces a set of problems absent from the single-aggregator case.

The first such problem is maintaining global model consistency across aggregator instances. Each aggregator receives updates from only a subset of clients, producing a sub-global model accumulated



(a) Client utilization for Flower as participating clients scale from 100 to 895, configured with a 95% client quorum and 10MB payloads. Utilization declines monotonically from 0.85 to below 0.12, at which point Flower fails entirely.



(b) Client utilization for Papaya collapses from 0.41 to near zero as client load grows from 500 to 8,000.

Figure 1: Scalability limitations of existing FL frameworks. Both Flower and Papaya suffer severe client underutilization at scale, with clients spending the vast majority of their time idle rather than training.

from the subset of clients. The sub-global models produced by individual aggregators would require an additional step to produce a consistent global model that incorporates updates from all clients, a coordination requirement inherent to any system that partitions shared state across multiple nodes [17].

The second challenge with aggregator scaling is model staleness. Due to the differences in the subset of updates that aggregators individually receive, the model version a client trains on may diverge from the current global model. This divergence causes a subset of clients to take longer to converge as staleness increases [36]. The third challenge with aggregator scaling is scheduling client updates

Design Insight 1: Scaling FL aggregation requires more than replicating a single aggregator; consistency, staleness, scheduling, and state propagation costs multiply with each instance added. An explicit step is further required to merge sub-global models into a consistent global model. This motivates a two-tiered architecture in which coordination is bounded within each tier and a dedicated aggregation step merges sub-global models into a single global model.

across aggregators, which is difficult for two reasons. First, aggregation is computationally expensive, so a scheduler that ignores load can bottleneck a single aggregator even while others sit idle. Second, incoming client updates must be placed on an aggregator holding the current global model rather than an older one, to avoid wasting the client’s training effort on a model the system has already moved past. The fourth challenge with aggregator scaling is propagating a common state across aggregators following aggregation. The updated global model, along with server-side optimizer state such as momentum or adaptive learning-rate terms maintained alongside the model, must be propagated to every aggregator [27]. The synchronization of the global model and server-side optimizer state must occur throughout the FL process, making it a persistent challenge. These four challenges illustrate that naively scaling single-aggregator systems horizontally in response to client load is insufficient for maintaining model consistency and client performance. The FL system architecture must therefore be re-designed from a single-aggregator model to one with FL-specific coordination, synchronization, and scheduling.

Aggregators in FL systems are deployed on always-on, on-demand cloud instances [11, 26]. To quantify the cost, we measure a median per-round aggregator time of 732 seconds on a p3.2xlarge instance and extrapolate to the 1,198 rounds required for FedAvg convergence on ResNet-18 trained on CIFAR-100 [31], yielding a projected training duration of approximately 244.5 hours. Figure 2 shows the resulting aggregator cost under three scenarios: on-demand pricing, ideal spot pricing assuming no preemptions, and spot pricing without fault tolerance where each preemption forces a full job restart. Spot instances offer a substantially cheaper alternative to on-demand instances [32, 35], being up to 88% cheaper [16]; however, spot instances can be preempted at any time by the cloud provider and come with no availability guarantees [10, 30]. While ideal spot pricing reduces aggregator cost by 88.7% over on-demand, preemptions without fault tolerance force full job restarts, inflating the realized spot cost to only 13.5% cheaper than on-demand, nearly erasing the spot pricing advantage altogether. Therefore, realizing the full cost benefits of spot instances requires fault tolerance mechanisms that preserve training state across preemptions [4, 28]. While techniques such as checkpointing and instance migration are commonly adopted for this purpose, each presents fundamental limitations when applied to FL aggregation systems. We identify three such limitations below. The first limitation is to do with Checkpointing, it is a commonly adopted FT technique that persists application state to durable storage, enabling recovery following a failure. In the context of FL aggregation, a naive checkpointing strategy saves the global model state after every aggregation update. However, as the number of aggregation updates grows, the cumulative I/O

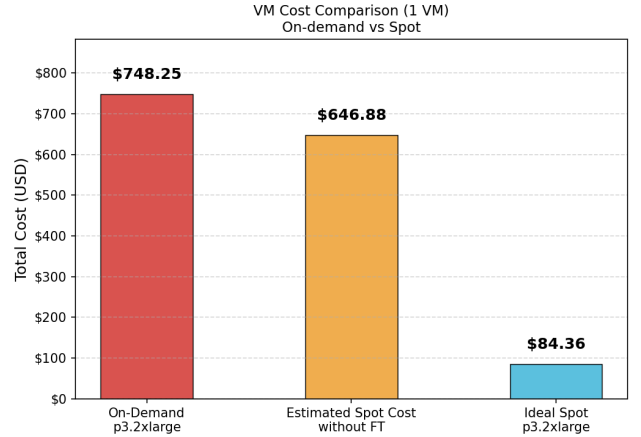


Figure 2: Total aggregator cost to train ResNet-18 on CIFAR-100 to convergence (1,198 rounds [31]) on a single p3.2xlarge instance. On-demand pricing at \$3.06/hr costs \$748.25 for 244.5 hours of uninterrupted training. Spot preemptions, in the absence of fault tolerance, force full job restarts and inflate training cost to \$646.88 at \$0.345/hr, a 7.6x overhead over the ideal spot cost of \$84.36 assuming uninterrupted execution.

Design Insight 2: Checkpointing, instance migration, and provider termination signals are each insufficient for FL aggregation on spot infrastructure. Effective fault tolerance requires decoupling aggregator state from instance lifecycle and acting on failure likelihood independently of provider-issued signals.

overhead of checkpointing every update grows proportionally. At scale, this overhead becomes a significant bottleneck, stalling the aggregator and degrading training throughput.

The second limitation is to do with migration, it is a FT technique that transfers the state of a failed instance to a newly provisioned replacement, allowing the system to resume operation. In the context of FL aggregation, this involves transferring the in-progress sub-global model, optimizer state, and client update buffer to a replacement aggregator following a spot preemption. However, cloud VM provisioning is not instantaneous; prior work has demonstrated that newly provisioned VMs incur substantial startup latency [8, 34]. During this provisioning window, the aggregator is unavailable, and for a system processing a large volume of client updates, this downtime results in lost updates, stalled training and increased time to reach target accuracy.

Finally Spot VM lifetimes are non-deterministic and have no prior knowledge of when preemption will occur. Although cloud providers issue a termination signal prior to reclamation, the notice period varies considerably across providers; AWS issues a two-minute warning [1] while GCP issues a 30-second warning [7]. The state transfer overhead inherent to FL aggregation systems may exceed the available notice window, rendering provider termination signals an insufficient basis for guaranteeing successful migration prior to instance reclamation.

Design Insight 3: Scaling the aggregator is necessary to sustain throughput under growing client load, yet multiple on-demand instances compound infrastructure cost. Spot instances present a viable path to containing this cost, but as existing fault tolerance mechanisms are insufficient to guard against preemptions, realized spot costs inflate to near on-demand levels, erasing the pricing advantage entirely. A framework that natively co-designs horizontal scaling with spot-aware fault tolerance is therefore required to achieve both performance and cost efficiency.

3 SpotLight System Overview

SpotLight enables federated learning to scale elastically on spot infrastructure without sacrificing aggregator performance or cost efficiency. This section overviews the architecture of SpotLight (§3.1) and the three design principles behind it (§3.2): *two-tiered aggregation* that scopes each aggregator to its assigned clients and combines partial models into a consistent global model, *proactive fault tolerance* that migrates workloads and checkpoints aggregator state upon detecting failure likelihood, and *elastic spot provisioning* that treats scalability and fault tolerance as co-equal concerns to fully exploit spot infrastructure.

3.1 SpotLight Architecture

Fig. 3 shows the architecture of SpotLight. The system comprises three main components: the **Resource Manager (RM)**, the **aggregation layer**, and the **Performance Manager**. The aggregation layer consists of **Sub-Global Aggregators**, **Global Aggregators**, and **Failover Edge Aggregators**, which extend the aggregation hierarchy to the edge when cloud aggregators are unavailable. All external client connections are fronted by Envoy, and MongoDB persists coordination state shared across all components.

The RM is itself horizontally scalable, deployed as multiple Kubernetes instances, and internally comprises four subcomponents. The **Scheduler** manages client membership, assigns clients to Sub-Global Aggregator cohorts, and schedules incoming updates. The **Scaler** provisions and deprovisions Sub-Global and Global Aggregators in response to signals from the Performance Manager, forming a closed loop that drives elastic scaling. The **Migrator** monitors aggregator failure likelihood and, upon detecting an at-risk instance, determines an appropriate migration target, either a healthy cloud aggregator or a Failover Edge Aggregator, and triggers workload migration and state checkpointing on the at-risk instance. The **Leader Elector** runs Raft consensus across RM instances to elect a single leader with sole authority over scaling and migration decisions, allowing the RM to scale horizontally without becoming a single point of failure.

SpotLight Execution Flow. Clients submit updates through Envoy to the RM, where the Scheduler assigns each update to a Sub-Global Aggregator cohort. Sub-Global Aggregators accumulate updates from their assigned clients and produce sub-global models, which Global Aggregators merge into a single consistent global model. Concurrently, the Performance Manager observes load across both aggregation tiers and signals the Scaler to provision or deprovision aggregators as demand shifts. When the Migrator detects elevated

failure likelihood on an aggregator, the elected leader directs workload migration to a healthy target and checkpoints the at-risk instance’s current state. If no healthy cloud aggregators are available, a Failover Edge Aggregator assumes the role. The RM redelivers any in-flight updates to the replacement, and training continues without restarting.

3.2 Three Design Principles

We further elaborate on the three design principles of SpotLight, explaining how each addresses the insights identified in §2.

Two-tiered aggregation. Scaling aggregators increases coordination and synchronization overhead. SpotLight manages this through a two-tiered aggregation structure in which each Sub-Global Aggregator handles only its assigned subset of clients, and Global Aggregators receive only a fixed number of sub-global models regardless of total client count. This bounds the coordination scope of each tier independently, allowing both tiers to scale without overhead growing proportionally with the number of clients or aggregators. Global Aggregators then merge the fixed set of sub-global models into a single consistent global model, providing an explicit aggregation step that scales independently of client count.

Proactive fault tolerance. Naive fault tolerance mechanisms that rely on provider termination signals, periodic checkpointing, or restarting lost work are insufficient for FL aggregation on spot infrastructure. SpotLight instead uses a Kaplan-Meier survival model [13] derived from publicly available spot VM lifetime traces [35] to estimate failure likelihood ahead of preemption, triggering workload migration and state checkpointing before an instance is lost. This ensures that aggregation continues without losing any in-flight updates.

Elastic spot provisioning. On-demand instances make sustained horizontal scaling economically prohibitive. SpotLight provisions its aggregation layer on spot instances, but the cost advantage of spot can only be realized if the system can absorb aggregator disruptions without stalling throughput. Scalability and fault tolerance are therefore co-equal concerns in SpotLight, working together simultaneously to sustain aggregator throughput and preserve the cost efficiency of spot infrastructure at scale

4 System Components

Building on the three design principles established in §3.2, this section describes how SpotLight realizes them in practice. The aggregation tiers realize two-tiered aggregation through independently scalable Sub-Global and Global Aggregators. The Resource Manager handles scheduling, orchestrates scaling across both tiers, and migrates workloads before preemption occurs, preserving training continuity and the cost efficiency of spot infrastructure. We begin with the update lifecycle (§4.1), which establishes the end-to-end path a client update follows through the system, then describe each component in detail.

4.1 Update Lifecycle

In SpotLight, client updates flow through two aggregation tiers, accumulating into sub-global models before being merged into the global model. Clients maintain persistent gRPC connections

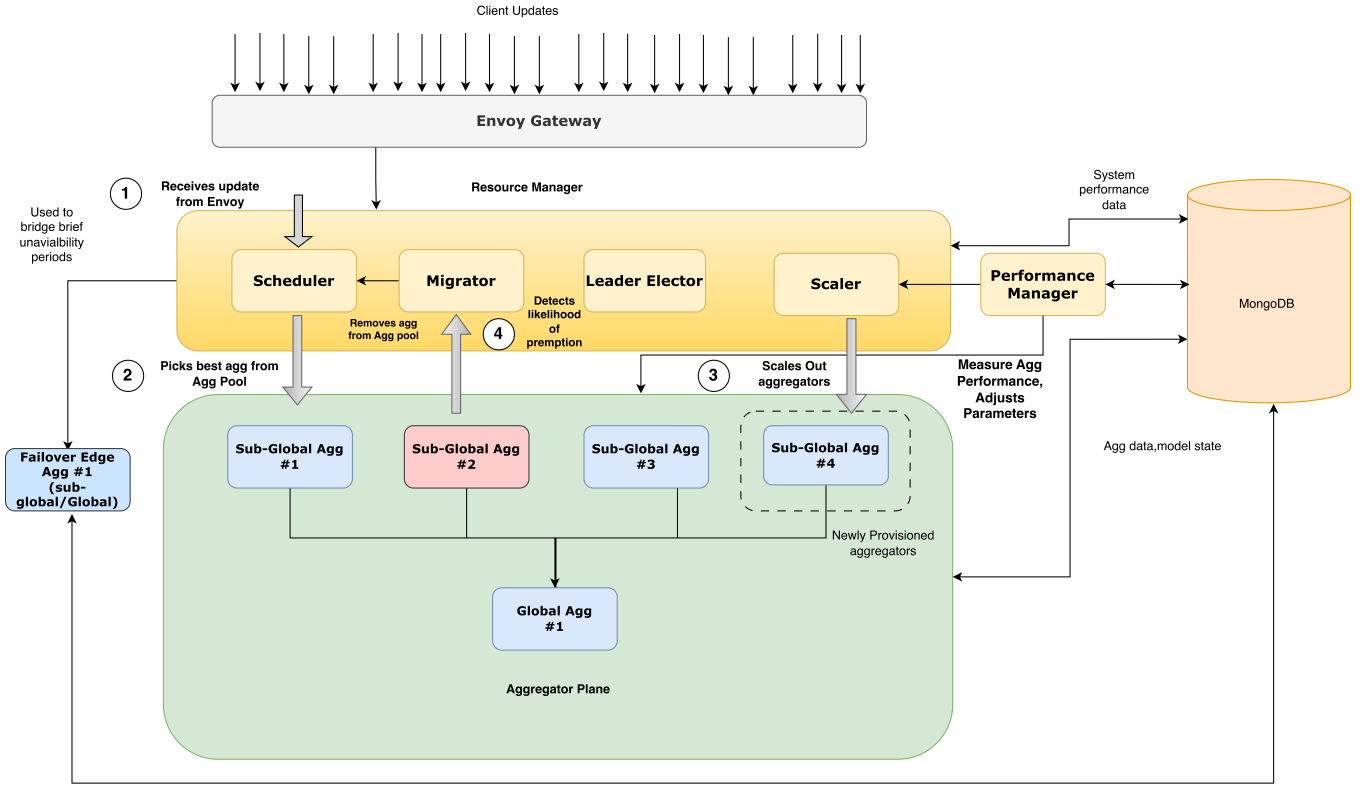


Figure 3: SpotLight architecture. Client updates enter through the Envoy Gateway and are placed by the Scheduler onto Sub-Global Aggregator instances (1, 2). The Scaler provisions new aggregator instances in response to signals from the Performance Manager (3). The Migrator continuously evaluates preemption risk and removes at-risk instances from the Scheduler’s candidate pool (4). Failover Edge Aggregators are utilized temporarily when aggregators are unavailable

to Envoy, which routes them to the Resource Manager. The RM forwards client updates to the appropriate Sub-Global Aggregator asynchronously in the background, decoupling client-facing latency from aggregator throughput. The Sub-Global Aggregator accumulates updates in an in-memory buffer and triggers aggregation once the floor threshold is met. The resulting sub-global model is pushed to the Global Aggregator, which incorporates it into the global model. Sub-Global Aggregators pull the updated global model and propagate it down the hierarchy to clients.

4.2 Two-Tiered Aggregation

SpotLight distributes aggregation across two tiers, allowing each tier to scale independently as client participation grows.

Aggregators operate at two distinct points in the hierarchy with different responsibilities. Sub-Global Aggregators receive updates from the RM, aggregate them into a sub-global model, and ensure clients receive the latest available model. Global Aggregators receive sub-global models from all Sub-Global Aggregators and merge them into a single global model. Both tiers adopt FedBuff’s [22] buffered aggregation model, but use the buffer as a floor rather than a strict barrier, triggering aggregation as soon as the minimum threshold is met, avoiding stalls from incomplete batches and improving aggregator throughput. Sub-Global Aggregator workload

scales directly with client participation and can vary significantly as clients join or leave training, while Global Aggregator workload is bounded by the number of Sub-Global Aggregators, making it more stable. This difference in workload characteristics motivates distinct scaling policies at each tier.

Scaler elastically scales both aggregation tiers in response to client load and system performance. The scaling of Sub-Global and Global Aggregators is managed by the RM. The scaling policy used by SpotLight for Sub-Global Aggregators utilizes client participation and utilization to scale horizontally, while Global Aggregators use memory utilization to scale. The difference in scaling signal between the two tiers is motivated by the workload characteristics of each tier. The algorithm used for scaling Sub-Global Aggregators is described in Algorithm 2. The Scaler utilizes this algorithm along with real-time measurements and scaling parameters obtained from the Performance Manager to scale Sub-Global Aggregators. For Global Aggregators, SpotLight internally implements the Kubernetes HPA algorithm, with scaling parameters likewise supplied by the Performance Manager.

Performance Manager dynamically adjusts the scaling parameters consumed by the Scaler across both aggregation tiers in response to observed system performance. The Performance Manager implements a Proportional-Integral-Derivative (PID) controller that

monitors aggregation throughput and reacts to changes between consecutive control cycles. A fixed setpoint is avoided as FL workloads operate under variable client participation, making a stable throughput baseline difficult to establish without extensive tuning. Instead, the controller reacts to changes in throughput between cycles, tightening scaling parameters when throughput degrades and relaxing them when it improves. The error signal $e(t)$ and controller output $u(t)$ are defined in Equations 1 and 2. The PID parameters used in our deployment are listed in Table 1.

Scheduler places incoming client updates onto available Sub-Global Aggregators using a queue-aware placement policy, formally defined in Algorithm 1. In SpotLight, aggregation begins once the minimum number of client updates are present on the aggregator. The bursty nature of FL workloads means naive placement policies risk stalling aggregation by spreading updates thinly across aggregators, none of which reach the floor threshold. The Scheduler addresses this by selecting the two aggregators with the largest queues from the active pool, concentrating updates to trigger aggregation as early as possible. The selected aggregator's AFS is incremented on every scheduling decision and decays over time since the last selection. When aggregators are equally ranked by queue length, AFS serves as a tiebreaker, deprioritizing aggregators with more recent scheduling decisions in favor of those that have been selected less recently. The decision flow is illustrated in Figure 4.

Version Consistency As Sub-Global and Global Aggregators scale independently, concurrent aggregation events across tiers can produce divergent model versions, causing clients to train on stale or inconsistent state. SpotLight addresses this with a shared monotonic version counter in MongoDB, atomically incremented by all aggregators whenever a new aggregated model is produced. The version counter provides a consistent ordering of aggregation events across the hierarchy, ensuring that clients always receive the most recent model regardless of which aggregator serves it. All aggregators detect and recover stale in-memory state after restarts by comparing their local version against the shared counter, pulling the latest model from persistent storage when a discrepancy is found.

$$e(t) = T_{\text{current}} - T_{\text{previous}} \quad (1)$$

$$u(t) = K_p e(t) + K_i \cdot \text{clamp}(\gamma \cdot I_{t-1} + e(t) \cdot \Delta t, -I_{\max}, I_{\max}) + K_d \frac{de(t)}{dt} \quad (2)$$

Table 1: PID Controller Values

Parameter	Value
K_p	0.05
K_i	0.02
K_d	0.10
γ	0.97
$I_{\max}$	10.0

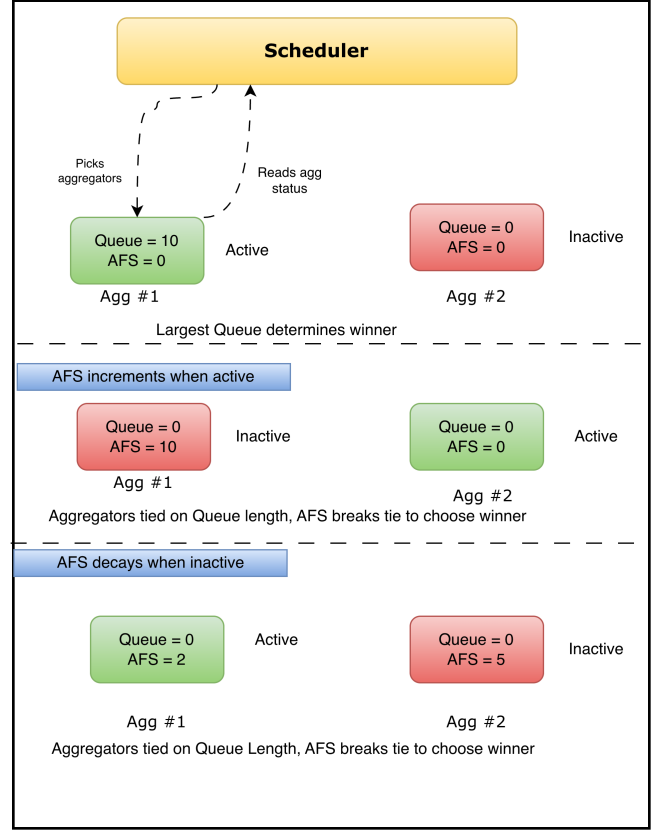


Figure 4: Scheduling algorithm. SpotLight primarily selects the aggregator with the largest queue to begin aggregation; when no aggregator has a clear lead, the Aggregation Fairness Score (AFS) breaks the tie. AFS measures how recently an aggregator was selected, with a higher score indicating heavier recent use. To avoid overloading any single aggregator, the scheduler favors aggregators that have not been picked recently, incrementing AFS each time an aggregator is chosen and decaying it exponentially over time.

4.3 Proactive Fault Tolerance

Spot instances can be reclaimed by the cloud provider at any time, making reactive recovery insufficient. SpotLight utilizes proactive fault tolerance (FT) techniques to mitigate the impact of spot pre-emptions on training progress.

Migrator is responsible for two key FT operations in SpotLight: workload migration and checkpoint triggering. The Migrator derives a Kaplan-Meier model [13] from publicly available lifetime traces [35] to estimate the likelihood of aggregator failure based on aggregator age. The Migrator tracks the age and preemption events of aggregators and operates in a closed loop with the Scheduler described in Section 4.2. The Migrator proactively detects aggregators at risk of failure and removes them from the candidate pool. The Scheduler then uses the updated pool to schedule incoming client updates on the available aggregators. When migration is not possible due to aggregator unavailability, the Migrator detects this

Algorithm 1 Scheduling Algorithm

Input: M : metrics $\rightarrow$ aggregator $\mapsto \langle q \rangle$, where q = queue size
Input: AFS: fairness score map (as of last update)
Input: AFS_{ts} : last-update timestamp map
Input: λ : decay factor

```

1: function EFFECTIVEAFS( $p, t$ )
2:   return  $\text{AFS}[p] \cdot \lambda^{t - \text{AFS}_{\text{ts}}[p]}$   $\triangleright$  Continuous decay
3: function ONPICK( $p, t$ )
4:    $\text{AFS}[p] \leftarrow \text{EFFECTIVEAFS}(p, t) + 1$   $\triangleright$  Refresh, then increment
5:    $\text{AFS}_{\text{ts}}[p] \leftarrow t$ 
6:  $C \leftarrow \text{MIGRATOR.GETSCHEDULABLEAGGREGATORS}$   $\triangleright$  Step 1: set pool
7:  $t \leftarrow \text{Now}$ 
8:  $Q_{\max} \leftarrow \max_{p \in C} M[p].q$   $\triangleright$  Step 2: rank by queue
9:  $\mathcal{T} \leftarrow \{p \in C : M[p].q = Q_{\max}\}$   $\triangleright$  Ties
10:  $p^* \leftarrow \arg \min_{p \in \mathcal{T}} \text{EFFECTIVEAFS}(p, t)$   $\triangleright$  Break with AFS
11:  $\text{ONPICK}(p^*, t)$   $\triangleright$  Step 3: update AFS
12: schedule on  $p^*$ 

```

and directs the Scheduler to trigger a Failover Edge Aggregator, which acts as a temporary bridge until a replacement aggregator is provisioned. The migration mechanism is shown in Fig. 5a and b.

Algorithm 2 Scaling Algorithm.

```

// Collect Inputs
1:  $\text{num\_updates} \leftarrow \text{GETNUMUPDATES}$ 
2:  $\text{current\_replicas} \leftarrow \text{GETCURRENTREPLICAS}$ 
3:  $\text{scale\_threshold}, \text{cooldown\_time} \leftarrow \text{GETFROMPERFORMANCEMANAGER}$ 
4:  $\text{util\_threshold} \leftarrow \text{Env.UTIL\_THRESHOLD}$ 

// Step 1: Client-Based Policy
5:  $\text{desired\_replicas} \leftarrow \lceil \text{num\_updates} / \text{scale\_threshold} \rceil$ 
6: if  $\text{desired\_replicas} > \text{current\_replicas}$  then
7:   Scale out to  $\text{desired\_replicas}$ 
8:    $\text{BEGINCOOLDOWN}(\text{cooldown\_time})$ 
9: else
10:  if  $\text{desired\_replicas} < \text{current\_replicas}$  then
11:    Scale in to  $\text{desired\_replicas}$ 
12:     $\text{BEGINCOOLDOWN}(\text{cooldown\_time})$ 
13:  else
14:    // Step 2: Utilization-Based Policy (fallback)
15:     $\text{current\_util} \leftarrow \text{training\_time} / (\text{training\_time} + \text{avg\_wait\_time} + \text{avg\_agg\_time})$ 
16:    if  $\text{current\_util} < \text{util\_threshold}$  then
17:       $\lambda_{\text{updates}} \leftarrow \text{GETUPDATERATE}$ 
18:       $\text{desired\_replicas} \leftarrow \lceil \lambda_{\text{updates}} / (\text{scale\_threshold} \cdot \text{util\_threshold}) \rceil$ 
19:      if  $\text{desired\_replicas} > \text{current\_replicas}$  then
20:        Scale out to  $\text{desired\_replicas}$ 
21:         $\text{BEGINCOOLDOWN}(\text{cooldown\_time})$ 
22:      else
23:        if  $\text{desired\_replicas} < \text{current\_replicas}$  then
24:          Scale in to  $\text{desired\_replicas}$ 
25:           $\text{BEGINCOOLDOWN}(\text{cooldown\_time})$ 

```

Checkpointing in SpotLight is triggered by the Migrator when an aggregator is identified as likely to be preempted. The aggregator persists its model state, in-buffer client contributions, and server-side optimizer state to the database. Optimizer state is saved conditionally based on the aggregation algorithm in use: stateless algorithms such as FedAvg require no optimizer state to be saved, while stateful algorithms such as FedAdam persist their server-side optimizer state alongside the model. This design reduces I/O overhead compared to periodic checkpointing, persisting state only when the Migrator determines recovery is likely to be needed.

Leader Elector protects the RM, which serves as the control plane for SpotLight, coordinating scaling, migration, and scheduling across the entire system. A single RM instance would represent a critical point of failure; the RM therefore scales horizontally, introducing the need for consensus to prevent conflicting control plane decisions across instances. SpotLight uses the Raft consensus protocol [24] to maintain a single elected leader among RM instances at all times. The leader holds exclusive authority over scaling and migration decisions, pushing them to all follower instances, which use them as the basis for scheduling incoming client updates. When the leader fails, Raft elects a successor automatically, restoring full

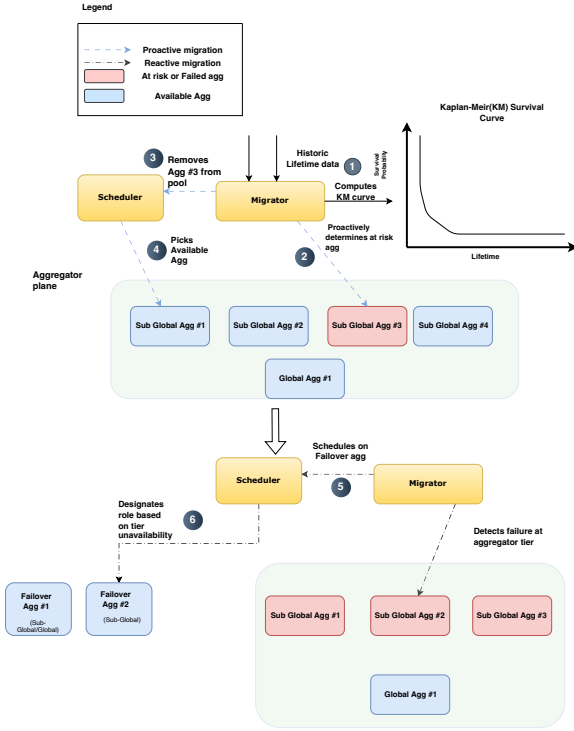


Figure 5: Migrator decision flow. The Migrator uses a Kaplan-Meier survival model [13] to compute survival probability from historic aggregator lifetime data, proactively identifying at-risk aggregators and removing them from the Scheduler’s candidate pool before failure occurs (top). When an aggregator tier fails and no aggregator in the pool is available, the Migrator detects the failure and directs the Scheduler to designate a Failover Aggregator based on which tier became unavailable (bottom).

control plane authority without interrupting scheduling operations across instances.

4.4 Elastic Spot Provisioning

SpotLight addresses two key challenges of FL systems deployed on spot infrastructure, variable client demand and spot preemption, through the coordinated operation of the components described in Sections 4.2 and 4.3. The Scaler and Performance Manager work in a closed loop to maintain aggregation throughput as client participation fluctuates, adjusting provisioning in response to observed system performance. Concurrently, the Migrator and Scheduler operate in a closed loop to handle spot preemptions: the Migrator continuously updates the pool of available aggregators based on predicted preemption risk, while the Scheduler places incoming updates on the aggregators most likely to trigger aggregation soonest within that pool. When no aggregator is available, Failover Edge Aggregators serve as a bridge. Working concurrently, these

components allow SpotLight to adapt to demand variability and infrastructure instability simultaneously, without manual intervention.

5 Implementation

5.1 SpotLight Implementation

SpotLight is implemented in Python across 19.2k lines of code, reflecting the breadth of its control plane, fault tolerance, and scheduling components, and is deployed on a Kubernetes cluster. SpotLight utilizes third-party libraries including Motor, an asynchronous MongoDB driver, gRPC for inter-component communication, NumPy for algorithm implementations, and the Kubernetes Python client for cluster management. SpotLight uses MongoDB for coordination, state management, and client lifecycle tracking. System components are deployed as StatefulSets, which provide stable network identities necessary for gRPC-based inter-component communication, and communicate exclusively over gRPC, chosen over REST for its HTTP/2-native multiplexing and bidirectional streaming, which eliminates per-request connection overhead on the critical path of persistent client-to-RM streams. All inbound client connections are routed through an Envoy gateway, selected for its native gRPC support and connection load balancing. SpotLight was hosted on Chameleon Cloud [14] during evaluation. The algorithms supported by SpotLight are listed in Table 2.

Table 2: Supported Aggregation Algorithms

Algorithm	Global Model Update
FedAvg [21]	$w^{(t+1)} = \sum_{k=1}^K \frac{n_k}{N} w_k^{(t)}$
FedBuff [22]	$w^{(t+1)} = \frac{\sum_{k \in B} n_k w_k^{(t)}}{\sum_{k \in B} n_k}$
FedNova [33]	$m^{(t+1)} = \beta_m m^{(t)} + (1 - \beta_m) \frac{\sum_k p_k (w^{(t)} - w_k^{(t)}) / r_k}{\sum_k p_k / r_k}$, $w^{(t+1)} = w^{(t)} - \eta_s m^{(t+1)}$
FedAdam [27]	$w^{(t+1)} = w^{(t)} - \eta \cdot \frac{\bar{m}^{(t)}}{\sqrt{\bar{g}^{(t)} + \epsilon}}$, $g = w^{(t)} - \bar{w}$
FedYogi [27]	$w^{(t+1)} = w^{(t)} - \eta \cdot \frac{\bar{m}^{(t)}}{\sqrt{\bar{g}^{(t)} + \epsilon}}$, $g = \bar{w} - w^{(t)}$

5.2 Spot Emulator

A key feature of spot instances is their variability in preemption behavior, driven by factors including region, availability zone, instance type, temporal market conditions and cloud providers [4, 18, 29]. This variability across configurations makes real-world evaluation of application behavior on spot instances difficult to reproduce consistently and cost-prohibitive at scale, as comprehensive coverage across regions and instance types requires extensive testing for long periods of time. Therefore, to evaluate SpotLight under realistic spot conditions, we developed the Spot Emulator, a high-fidelity trace-driven emulator that replays spot VM behavior using publicly available spot availability traces [16, 35]. The emulator is designed around two key goals. First, high fidelity: the emulator replays preemption events derived from real-world availability traces, ensuring emulated behavior reflects realistic spot market conditions. Second, controller-agnostic design: the modular architecture of the Spot Emulator allows it to support Spot VM behavior replay with different cloud services providers and deploy on all platforms with minimal changes to the emulator.

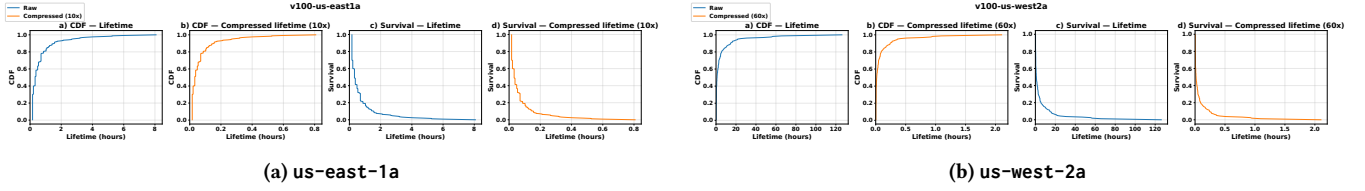


Figure 6: Lifetime and survival distributions for V100 spot instances, derived from availability traces in [35]. Panels (a) and (b) in each subfigure show the CDF of instance lifetime in raw and time-compressed form; panels (c) and (d) show the corresponding Kaplan-Meier survival curve. The time compression factor (10× for us-east-1a, 60× for us-west-2a) is a configurable parameter of the Spot Emulator, set independently per region to fit evaluation run-time constraints; it is not derived from the trace data itself. The Spot Emulator samples instance lifetimes from these distributions to replay preemption events during evaluation.

The lifetime distribution used to replay spot behavior is derived from the availability traces in [35]. The availability trace is used to derive a CDF, which allows the emulator to reliably replay spot instance lifetime dynamics. To ensure that evaluation cost is realistic, price traces obtained from [16] are used to generate realistic execution costs. As shown in Figure 6, survival probability decreases monotonically with instance lifetime, reflecting the inherent unreliability of spot instances over time.

The Spot Emulator consists of five components. The first is the *Distribution Extractor*, which filters the availability trace by region, availability zone, and OS, and derives the lifetime CDF from the filtered data. The second is the *Lifecycle Manager*, which samples a lifetime for each newly provisioned VM from the CDF within a configured range, excluding tail outliers that represent atypically short-lived or long-lived instances, and maintains an internal clock tracking the lifetime of all active spot VMs. The third component is the *Executor*, which terminates instances via the Kubernetes API once their assigned lifetime expires. The fourth component is the *Control Plane*, which manages the execution order of the other components and exposes configuration parameters to the user, including the sampling range and time compression factor. The fifth component is the *Cost Report Generator*, which uses price traces from [16] to produce accurate execution cost reports.

Execution begins when the Control Plane is configured with the lifetime trace source, sampling range, and time compression factor. The Control Plane invokes the Distribution Extractor to generate the lifetime CDF, which is passed to the Lifecycle Manager. The Lifecycle Manager samples a lifetime from the CDF within the configured range, assigns it to each pod, and tracks elapsed time via an internal clock. When a pod’s assigned lifetime elapses, the Lifecycle Manager invokes the Executor to terminate the pod.

The Spot Emulator’s logical model maps directly onto a Kubernetes cluster. The Cluster represents the datacenter, and the pods scheduled onto its worker nodes represent the Spot VMs, each assigned a lifetime and terminated once that lifetime expires. SpotLight is deployed on top of this emulated cluster, allowing it to be evaluated under realistic spot preemption behavior.

6 Evaluation Setup

SpotLight was evaluated on Chameleon Cloud [14], a platform designed for experimental systems research. SpotLight was deployed

on a five-node Kubernetes cluster, with node specifications listed in Table 3. MongoDB was hosted on a dedicated machine outside the cluster, with equivalent specifications to the host node. The evaluation focuses on two key aspects of the system: scalability and fault tolerance.

Table 3: Cluster Node Specifications

Specification	Host	Worker
CPU Model	Intel Xeon Gold 6126 @ 2.60GHz	—
vCPUs	48	16
Total RAM	187 GiB	28 GiB

6.1 Scalability Results

SpotLight is evaluated against Papaya [9], a production asynchronous FL system, as the baseline, chosen for its support of asynchronous aggregation at scale.

In order to emulate the behavior of FL clients, a testing platform was designed that follows the typical lifecycle of an FL client from registration to exit. The testing platform has configurable parameters listed in Table 4; the parameters allow the user fine-grained control over the behavior of the clients. The emulated client platform allows the user to emulate a large client population as well as the behavior of a typical FL client. The workload arrival patterns shown in Fig 7 are supported by our platform currently and can be easily added as well.

The scalability experiments use the parameters listed in Table 5 to load test the aggregators. The parameters were chosen to keep per-client execution time small, allowing experiments to focus mainly on aggregator scalability without getting bottlenecked by client side limitations.

The scalability experiments use the parameters listed below to measure the impact of scaling on aggregator performance.

1) Client Utilization. We define client utilization is defined as $\frac{T_{train}}{T_{train} + T_{aggregation} + T_{queue}}$, where T_{train} denotes local computation at the client, $T_{aggregation}$ denotes the time spent waiting for aggregation to complete, and T_{queue} denotes the time buffered at the Sub-Global Aggregator prior to aggregation. Higher utilization indicates a greater fraction of total time spent on useful local computation rather than coordination overhead.

2) No of aggregators This metric evaluates how the aggregation tier scales relative to client load across all supported algorithms.

Table 4: Emulated Client Configuration Parameters

Parameter	Description
Trace	Determines the number of clients active at any given time, governing the client population dynamics
No. of rounds	Number of training rounds the system is configured to run
Training time	Duration of local training at each client
Model size	Size of the FL model being trained

Table 5: Client Configuration for Scalability Experiments

Parameter	Value
No. of rounds	2
Training time	2s
Model size	10MB

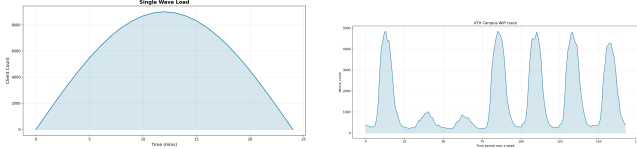
**(a) Single-wave synthetic trace. (b) KTH campus WiFi trace [25].**

Figure 7: Client arrival patterns used in the evaluation. The synthetic trace provides a controlled load shape for isolating scalability behavior, while the KTH trace captures the real-world variation in active client count over the course of a week, used to validate that the cost gains observed under the synthetic trace hold under realistic client arrival patterns.

A scalable design should demonstrate proportional and controlled growth in Sub-Global Aggregator instances as client population increases.

3) Queue Time. Queue time measures the duration each client spends waiting before its update enters aggregation, capturing contention at the aggregation layer. In a well-scaled system, queue time should remain bounded as client count grows.

4) Cost. Cost measures the total infrastructure expenditure required to operate SpotLight under varying client loads, expressed as instance-hours consumed by the aggregation tier over the duration of training.

5) Aggregator Throughput. Aggregator throughput measures the rate at which the aggregation tier processes client updates, expressed as updates aggregated per unit time.

6) Cost per Aggregation. Cost per aggregation measures the infrastructure expenditure incurred per completed aggregation event, computed as total instance-hour cost divided by the number of aggregations performed over the same run. This metric isolates whether cost efficiency holds as a function of work performed.

The scalability evaluation utilizes the two workload arrival patterns shown in Figure 7. The single-wave pattern provides a controlled, steadily increasing load, isolating scalability behavior from real-world variation. The KTH WiFi trace [25] evaluates the system

Table 6: Median client utilization across all four aggregation algorithms under the single-wave trace. SL = SpotLight.

Algorithm	SL	PBudget	PMax
FedAvg	0.98	0.80	0.95
FedAdam	0.98	0.80	0.93
FedNova	0.98	0.80	0.95
FedYogi	0.98	0.80	0.94

Table 7: Average queue time across all four aggregation algorithms under the single-wave trace. SL = SpotLight.

Algorithm	SL	PBudget	PMax
FedAvg	0.67s	3.78s	0.71s
FedAdam	0.64s	3.78s	0.87s
FedNova	0.67s	3.77s	0.71s
FedYogi	0.64s	3.78s	1.04s

Table 8: Total infrastructure cost across all four aggregation algorithms under the single-wave trace. SL = SpotLight. Papaya-Budget is provisioned at equal cost to SpotLight in every case.

Algorithm	SL Cost	PBudget Cost	PMax Cost
FedAvg	\$14.43	\$14.43	\$22.70
FedAdam	\$11.99	\$11.99	\$22.44
FedNova	\$13.90	\$13.90	\$22.70
FedYogi	\$11.99	\$11.99	\$22.70

under realistic conditions, where the number of active clients varies due to diurnal patterns, validating that gains observed under the single-wave pattern hold under real-world client arrival behavior.

Papaya is not scalable by design therefore in order to evaluate the performance of Papaya against SpotLight fairly, the configurations of Papaya are determined below

1) Papaya-Budget. The number of aggregators is derived from SpotLight’s execution cost, comparing performance under an equal budget for both systems.

2) Papaya-Max. Papaya’s aggregators are scaled up until its client utilization and queue time approach SpotLight’s, quantifying the additional cost required to match SpotLight’s performance without elastic scaling.

SpotLight is first evaluated using the single-wave workload with the configurations above to establish baseline scalability behavior. The KTH trace is then used to evaluate the framework under realistic client arrival conditions, ensuring SpotLight is tested comprehensively across both controlled and real-world settings.

6.1.1 Single-Wave Workload. This subsection tests whether SpotLight’s scaling behavior holds despite the varying per-round overhead across aggregation algorithms, under the controlled, single-wave synthetic trace shown in Fig 7a, evaluated across FedAvg, FedAdam, FedNova, and FedYogi.

Client Utilization. As summarized in Table ??, SpotLight sustains a median utilization of 0.98 across all four algorithms (Fig 8a, 9a, 10a, 11a). Papaya-Budget is similarly stable but at a lower ceiling, holding

a median of 0.80 in all four cases (Fig 8e, 9e, 10e, 11e); this flat curve indicates a fixed capacity limit rather than a bottleneck that worsens with client count. Papaya-Max varies more by algorithm, holding a median of 0.95 under FedAvg and FedNova, 0.93 under FedAdam, and 0.94 under FedYogi.

Client Utilization. As summarized in Table 6, SpotLight sustains a median utilization of 0.98 across all four algorithms (Fig 8a, 9a, 10a, 11a). Papaya-Budget is similarly stable but at a lower ceiling, holding a median of 0.80 in all four cases (Fig 8e, 9e, 10e, 11e); this flat curve indicates a fixed capacity limit rather than a bottleneck that worsens with client count. Papaya-Max varies more by algorithm, holding a median of 0.95 under FedAvg and FedNova, 0.93 under FedAdam, and 0.94 under FedYogi.

Queue Time. As summarized in Table 7, SpotLight’s average queue time stays between 0.64s and 0.67s across all four algorithms (Fig 8b, 9b, 10b, 11b), with a tightly clustered distribution in each case (Fig 8c, 9c, 10c, 11c). Papaya-Budget is consistently the slowest of the three, averaging between 3.77s and 3.78s in every algorithm, with almost no distribution overlap against the other two systems. Papaya-Max averages 0.71s under both FedAvg and FedNova, 0.87s under FedAdam, and 1.04s under FedYogi, the widest gap to SpotLight of the four algorithms, with a distribution tail extending to 4.1s.

Cost. SpotLight and Papaya-Budget are held to equal cost by construction in every algorithm (Fig 8d, 9d, 10d, 11d), so the cost comparison reduces to how much more Papaya-Max must spend to approach SpotLight’s utilization and queue time. As shown in Table 8, Papaya-Max costs \$22.70 under FedAvg against SpotLight’s \$14.43, and \$22.70 under FedNova against SpotLight’s \$13.90, both bringing it reasonably close to SpotLight’s utilization and queue time. Under FedAdam and FedYogi, Papaya-Max costs \$22.44 and \$22.70, respectively, against SpotLight’s \$11.99 in both cases, yet still falls short of SpotLight on both metrics. Across all four algorithms, matching SpotLight’s performance without elastic scaling requires Papaya-Max to spend nearly double SpotLight’s cost or more, and that additional spend does not fully close the gap under FedAdam or FedYogi.

6.1.2 KTH Campus WIFI Trace. The KTH campus wifi trace is utilized in order evaluate the SpotLight under real world conditions. The scalability of SpotLight with the KTH trace is evaluated with 2 algorithms, the first being FedAvg for stateless and FedAdam for stateful algorithms. Papaya is configured with the 5 aggregator, the same configuration that was followed with Papaya-Max in the previous section 6.1.1.

FedAvg FedAdam

Across both aggregation algorithms, SpotLight outperforms Papaya on cost, cost per aggregation, median throughput, and client utilization on the KTH trace.

FedAvg. Figure 12 shows SpotLight’s replica count tracking client demand over the trace. Over a projected 168-hour run (Figure 14), SpotLight costs \$1,298.68 compared to \$2,570.40 for Papaya, a reduction of approximately 49.5%. This lower cost comes without sacrificing performance: as shown in Figure 15, SpotLight sustains higher throughput than Papaya throughout the trace, with a median of 57.40 updates/sec against Papaya’s 48.26 (Figure 16), roughly

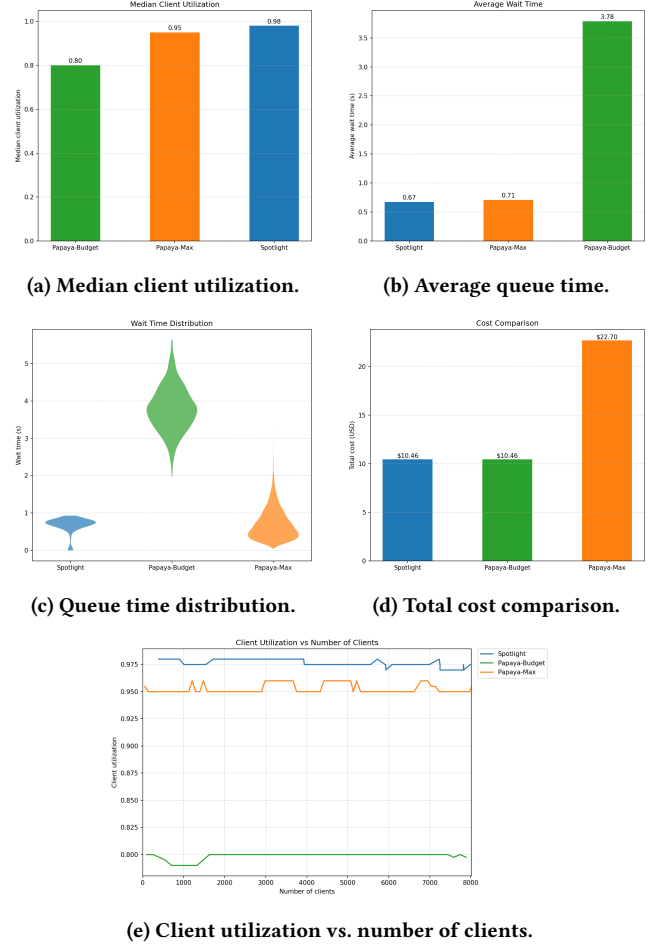


Figure 8: FedAvg results: client utilization, queue time, and cost under the single-wave trace.

1.19x higher, while spending only \$0.11 per aggregation versus Papaya’s \$0.24 (Figure 17), a 2.18x reduction. Client utilization follows the same pattern (Figure 13): SpotLight remains consistently above 0.97, while Papaya fluctuates more, dipping as low as approximately 0.95 at several points during the trace.

FedAdam. The same pattern holds under FedAdam, though the balance shifts toward throughput. Figure 18 shows SpotLight’s replica count over the trace, never dropping below two. SpotLight’s projected 168-hour cost is \$2,010.40 against Papaya’s \$2,570.40 (Figure 20), a 21.8% reduction, while Figure 21 shows SpotLight sustaining a wider throughput advantage than under FedAvg, with median throughput reaching 97.78 updates/sec compared to Papaya’s 48.26 (Figure 22), roughly 2.03x higher. Cost per aggregation falls to \$0.05 for SpotLight versus \$0.24 for Papaya (Figure 23), a 4.8x reduction. Client utilization under FedAdam is notably stable for SpotLight, holding essentially flat at approximately 0.99 for the full duration shown (Figure 19), while Papaya’s utilization continues to fluctuate between roughly 0.95 and 0.98.

SpotLight outperforms Papaya consistently across all metrics. The performance and cost efficiency gains between SpotLight and



Figure 9: FedAdam results: client utilization, queue time, and cost under the single-wave trace.

Papaya are driven by two key components in the system: the Scaler and the Scheduler. An even distribution of updates leads to queuing delays under the bursty nature of FL workloads, since it assigns updates without regard to which aggregators are closer to triggering aggregation. SpotLight’s load-aware scheduling algorithm instead uses aggregator state to place incoming updates on the aggregators most likely to begin aggregation soonest, reducing these delays, consistent with the wait-time results in Section 6.1.1. The Scaler, meanwhile, adds aggregators in response to client load, while the Scheduler exploits that added capacity to sustain high throughput. Together, the Scaler and Scheduler allow SpotLight to meet quickly client demand while keeping overall execution cost lower than Papaya configured for peak load under the max configuration.

While SpotLight outperforms Papaya under both algorithms, the magnitude of that advantage, particularly in overall cost, differs between them: running SpotLight with FedAdam costs 54.8% more than with FedAvg for the same workload, a gap driven by FedAdam’s stateful design. Because the aggregator must retain state throughout the aggregation process, memory contention becomes a bottleneck: during evaluation, this contention caused timeouts and



Figure 10: FedNova results: client utilization, queue time, and cost under the single-wave trace.

other performance issues when FedAdam ran with as few as one aggregator. SpotLight was therefore configured with a minimum of two aggregators for FedAdam, in contrast to the single-aggregator minimum used elsewhere. This shift is reflected across the entire trace, not just at the floor: as shown in Figure 24, FedAvg spends 96% of its runtime at one or two replicas, while FedAdam spends 59% of its runtime at three or four, a difference directly reflected in the cost gap between the two algorithms.

6.2 Fault Tolerance

To evaluate the fault tolerance of SpotLight, an ablation study was conducted on CIFAR-10 [15], training 10 clients under both IID and non-IID data distributions, with the non-IID split generated using a Dirichlet distribution with $\alpha = 1.2$. The Spot Emulator described in Sec 5.2 was used to inject Spot VM preemption while the FL process was occurring. The Spot Emulator was configured to use us-east1a-v100 traces, shown in Fig 6a, with a compression ratio of 10x. The fault tolerance policy was varied across runs in order to

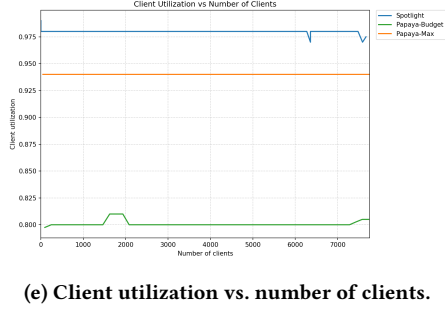
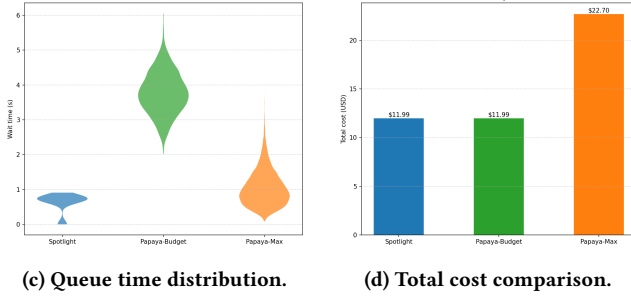
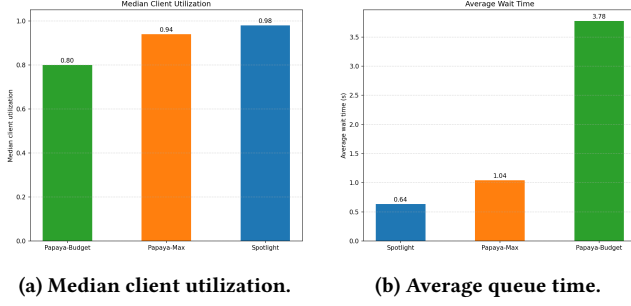


Figure 11: FedYogi results: client utilization, queue time, and cost under the single-wave trace.

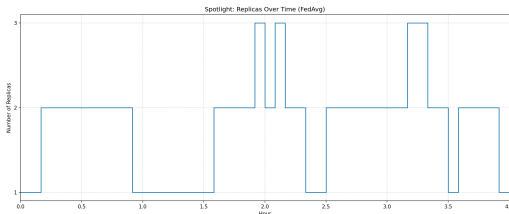


Figure 12: Number of SpotLight replicas over time under FedAvg, tracking the client trace shown in Figure 7b.

isolate the individual strengths of each policy. The configurations are described later in this section.

Configurations. Table 9 summarizes the ten configurations evaluated. **Spotlight-UB** is an upper-bound configuration in which no faults were injected. **Spotlight-LB** is a lower-bound configuration in which fault tolerance was disabled entirely, so faults were injected but never recovered from. **Naive-CP** checkpoints the model whenever a checkpoint is available, without any coordination on

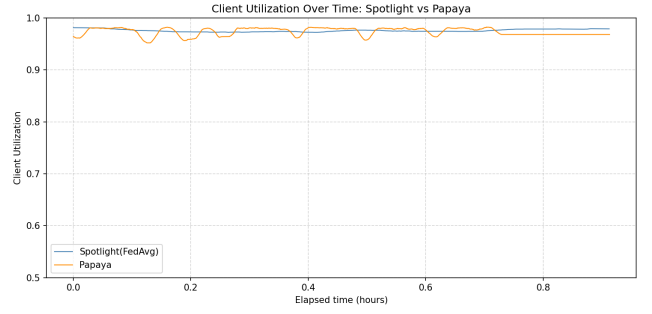


Figure 13: Client utilization over time, SpotLight vs. Papaya, on the KTH trace under FedAvg.

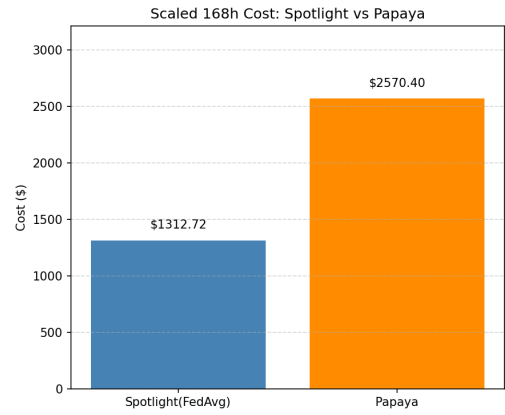


Figure 14: Projected total cost of SpotLight vs. Papaya over 168 hours (one week) on the KTH trace under FedAvg.

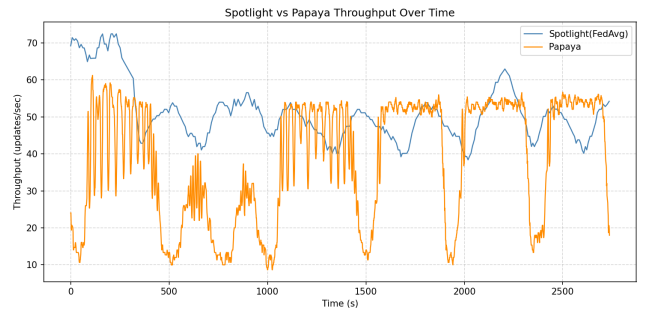


Figure 15: SpotLight vs. Papaya throughput over time on the KTH trace under FedAvg.

timing or placement. **Naive-migration** migrates the aggregator's state to a replacement node only after the aggregator has already failed. **Naive-migration+CP** combines naive checkpointing and naive migration, without failover aggregator offloading. **Naive-Full** extends Naive-migration+CP with failover aggregator offloading, so that checkpointing, migration, and offloading are all decided naively. **KM-CP**, **KM-migration**, and **KM-migration+CP** apply

SpotLight: Title is TBD

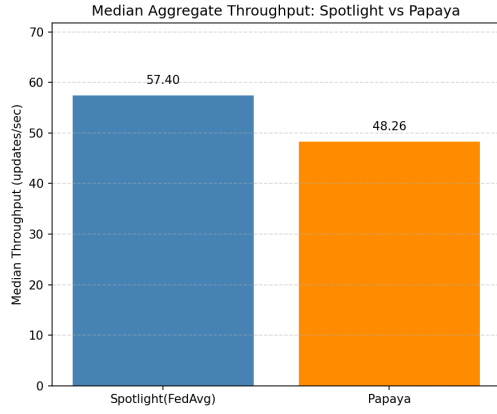


Figure 16: Median aggregate throughput, SpotLight vs. Papaya, on the KTH trace under FedAvg.

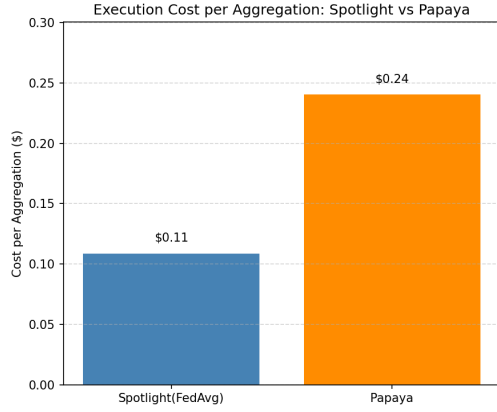


Figure 17: Execution cost per aggregation, SpotLight vs. Papaya, on the KTH trace under FedAvg.

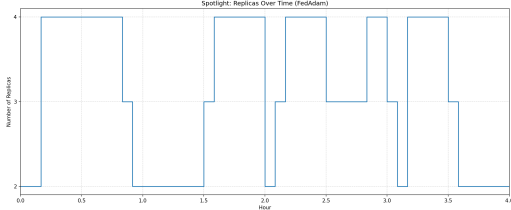


Figure 18: Number of SpotLight replicas over time under FedAdam, tracking the client trace shown in Figure 7b.

SpotLight’s KM configuration to checkpointing, migration, or both, respectively, without failover aggregator offloading. **KM-Full** is SpotLight’s default configuration, in which the KM configuration governs checkpointing, migration, and failover aggregator offloading together.

KM-Full reaches 81.52%, the closest of any fault-tolerant configuration to the fault-free upper bound of 84.34%, outperforming the

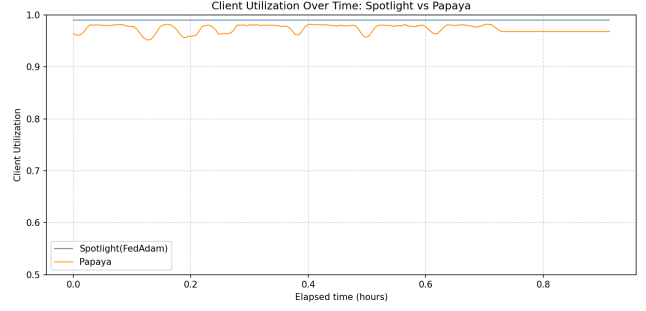


Figure 19: Client utilization over time, SpotLight vs. Papaya, on the KTH trace under FedAdam.

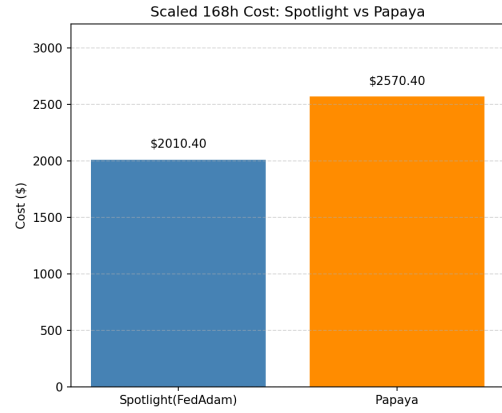


Figure 20: Projected total cost of SpotLight vs. Papaya over 168 hours (one week) on the KTH trace under FedAdam.

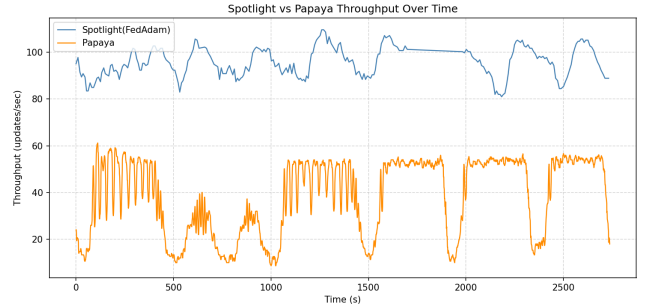


Figure 21: SpotLight vs. Papaya throughput over time on the KTH trace under FedAdam.

next-best configuration, Naive-migration (71.80%), by 9.7 percentage points. In both the IID and non-IID settings shown in Fig 25 and Fig 26, KM-Full is the best-performing configuration among those that recover from faults, consistent with the KM configuration scheduling recovery decisions ahead of Spot preemption.

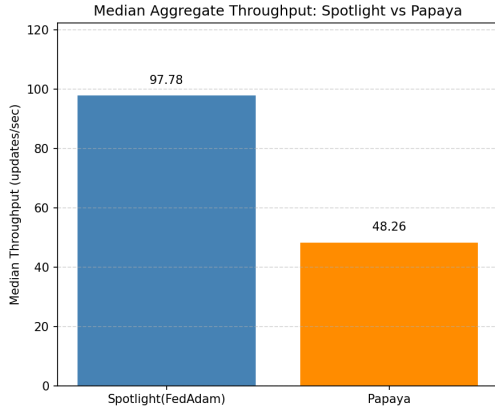


Figure 22: Median aggregate throughput, SpotLight vs. Papaya, on the KTH trace under FedAdam.

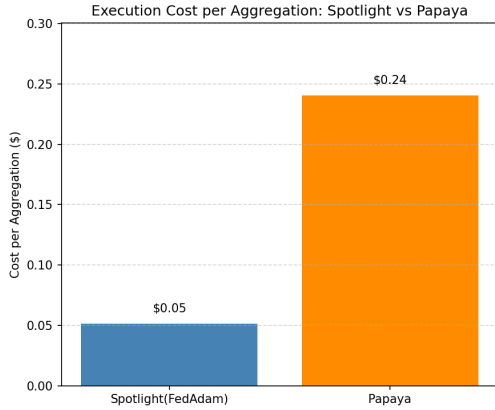


Figure 23: Execution cost per aggregation, SpotLight vs. Papaya, on the KTH trace under FedAdam.

The results above establish that SpotLight’s full KM-driven configuration achieves the highest accuracy among fault-tolerant configurations in both the IID and non-IID settings. The remainder of this section isolates the individual fault tolerance mechanisms, checkpointing, migration, and offloading, to explain the source of this improvement.

6.2.1 Checkpointing. Figure 27 compares checkpointing behavior between the KM configuration and Naive-CP. The KM configuration takes 83 checkpoints at an average of 784ms each, while Naive-CP takes 159 checkpoints at an average of 1706ms each, roughly half as many checkpoints at less than half the average checkpoint time. Despite taking fewer and faster checkpoints, the KM configuration still achieves higher accuracy than Naive-CP in both the IID and non-IID settings, as shown in Figures 25 and 26. This indicates that the KM configuration’s reduction in checkpointing overhead does not come at the cost of recovery quality.

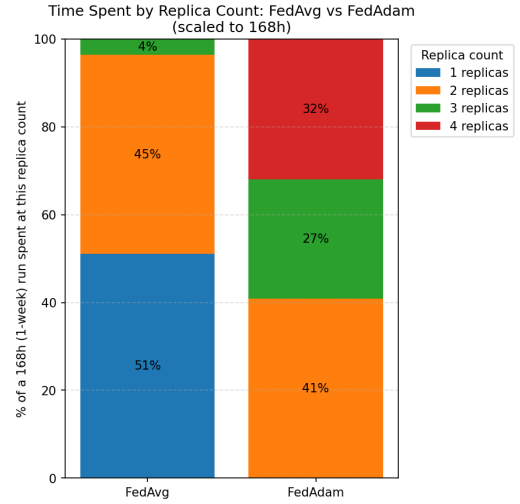


Figure 24: Percentage of a 168-hour (1-week) run spent at each replica count, SpotLight under FedAvg vs. FedAdam.

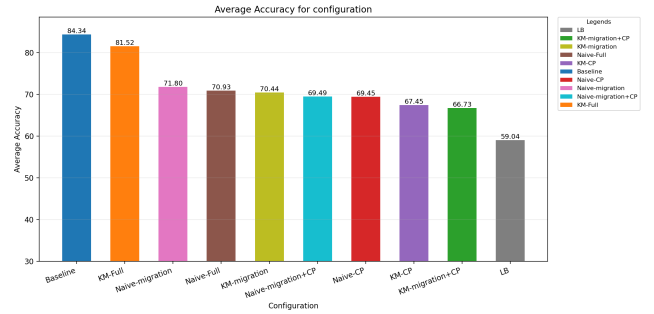


Figure 25: Average accuracy across fault tolerance configurations under IID client data (10 clients).

6.2.2 Migration. Figure 28 shows the number of client updates received under three configurations against the total client workload for a single wave, shown as a dashed line at 15,256. The no-migration configuration, in which no other aggregator is available to migrate to on failure, receives only 3,331 client updates, indicating substantial work loss. The migration configuration, in which the aggregator’s state is migrated to another available aggregator, recovers most of this gap, receiving 11,776 client updates. KM-Full, SpotLight’s full policy, additionally allows migration to failover nodes when no other aggregator is available, and comes within 0.01% of the single-wave total. This indicates that extending migration to include failover nodes closes nearly all of the remaining work loss left by aggregator-only migration.

6.2.3 Failover offloading.

7 Discussion

We now discuss three limitations of our approach: the assumption of uniform failure risk across aggregators, the cost of full-model

SpotLight: Title is TBD

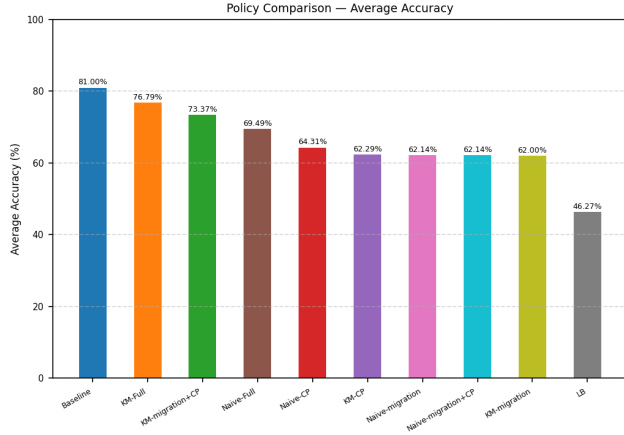


Figure 26: Average accuracy across fault tolerance configurations under non-IID client data (10 clients, Dirichlet $\alpha = 1.2$).

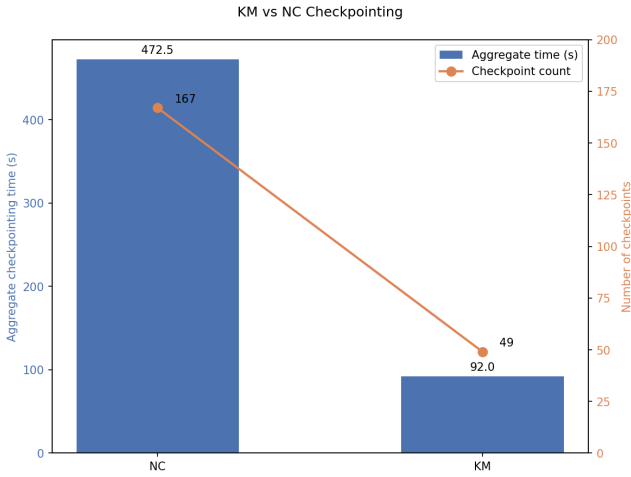


Figure 27: Average checkpoint time and checkpoint count: KM configuration vs. Naive-CP.

checkpointing under preemption, and the structural overhead of a static hierarchy.

Uniform failure risk across zones. SpotLight assumes that all aggregators are deployed within a single zone, under which migrating an aggregator to a healthy replica incurs low overhead. This assumption breaks down once aggregators are spread across multiple zones. The migrator currently treats any available aggregator as an equally valid migration target, but preemption rates differ substantially by zone, as shown in Figure 29. If the target zone is itself high-risk, migrating to it is a poor tradeoff: the aggregator may be preempted again shortly after, and each migration already carries non-trivial cost from cross-zone bandwidth limits and the latency of re-synchronizing FL state. A zone-aware migrator would need



Figure 28: Received client updates per configuration against the single-wave workload total. KM-Full corresponds to the migration-edge configuration, which extends migration to edge nodes.

Table 9: Fault tolerance ablation configurations

Configuration	Description
Spotlight-UB	No faults injected (upper bound)
Spotlight-LB	Faults injected, no recovery (lower bound)
Naive-CP	Checkpoint on availability only
Naive-migration	Migrate only after aggregator failure
Naive-migration+CP	Naive migration combined with naive checkpointing, no failover offloading
Naive-Full	Naive checkpointing + naive migration + failover aggregator offloading
KM-CP	KM checkpointing only
KM-migration	KM migration only
KM-migration+CP	KM migration and checkpointing, no failover offloading
KM-Full	KM checkpointing + KM migration + failover aggregator offloading, with the KM configuration governing all FT decisions

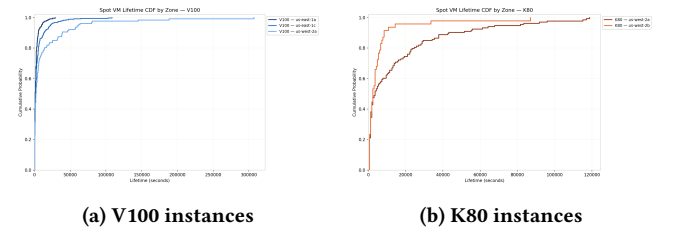


Figure 29: Kaplan-Meier estimates of spot VM lifetime by availability zone, computed from traces released with [35].

to weigh a candidate target’s preemption risk against this migration cost, rather than swapping to the first available aggregator regardless of its zone.

Full-model checkpointing under preemption. SpotLight checkpoints the full model at each checkpoint trigger, rather than checkpointing only the parameters that changed since the last checkpoint, an approach shown to reduce checkpoint write bandwidth for large models such as production recommendation models that can reach terabytes in size [5]. This approach works by storing a differential checkpoint relative to a baseline checkpoint, reducing what needs to be written each interval. However, it depends on that baseline being durably persisted; reconstructing the model requires reading both the baseline and the latest delta. On a spot instance, the baseline itself may not finish writing before the instance is preempted, in which case every delta computed against it becomes unusable. This makes differential checkpointing, despite being effective for large models on stable clusters, unsuitable for our setting without further changes.

Structural cost of a static hierarchy. SpotLight maintains a hierarchical aggregation structure regardless of the number of active aggregators, even though the hierarchy is only necessary once multiple aggregators must be coordinated. Figure ?? shows replica count over time under the KTH trace with FedAvg: the system operates with a single replica, for which a flat structure would suffice, for 23.2% of the observed trace, and scales to two or more replicas for the remaining 76.8% of the time. Once the system reaches a single-replica state, it could in principle collapse to a flat structure and improve cost efficiency, but this gain is offset by two costs: VM startup latency when scaling back into a hierarchy, and the synchronization overhead of migrating aggregator state across the structural change. Realizing this tradeoff in practice would require an optimization criterion that weighs the savings from operating flat against these switching costs before deciding to change structure.

8 Related Work

9 Conclusion

We present SpotLight...

References

- [1] Amazon Web Services. 2024. EC2 Spot Instance Interruption Notices. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-instance-termination-notices.html>. Accessed: 2024.
- [2] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. 2022. Flower: A Friendly Federated Learning Research Framework. *arXiv preprint arXiv:2007.14390* (2022).
- [3] Kallista A. Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingberman, Vladimir Ivanov, Chloé Kiddon, Jakub Konečný, Stefano Mazzocchi, H. Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. 2019. Towards Federated Learning at Scale: System Design. In *SysML*.
- [4] Jiangfei Duan, Ziang Song, Xupeng Miao, Xiaoli Xi, Dahua Lin, Harry Xu, Minjia Zhang, and Zhihao Jia. 2024. Parcae: proactive, liveput-optimized DNN training on preemptible instances. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation* (Santa Clara, CA, USA) (NSDI'24). USENIX Association, USA, Article 62, 19 pages.
- [5] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. 2022. Check-N-Run: a Checkpointing System for Training Deep Learning Recommendation Models. In *19th USENIX Symposium on Networked Systems Design and Implementation* (NSDI 22). USENIX Association, Renton, WA, 929–943. <https://www.usenix.org/conference/nsdi22/presentation/eisenman>
- [6] Kazim Ergun, Rishikanth Chandrasekaran, and Tajana Rosing. 2023. Federated Hyperdimensional Computing. *arXiv preprint arXiv:2312.15966* (2023).
- [7] Google Cloud. 2024. Spot VMs. <https://cloud.google.com/compute/docs/instances/spot>. Accessed: 2024.
- [8] Jianwei Hao, Ting Jiang, Wei Wang, and In Kee Kim. 2021. An Empirical Analysis of VM Startup Times in Public IaaS Clouds. In *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. 398–403. doi:10.1109/CLOUD53861.2021.00053
- [9] Dzmitry Huba, John Nguyen, Kshitiz Malik, Ruiyu Zhu, Mike Rabbat, Ashkan Yousefpour, Carole-Jean Vu, Hongyuan Zhan, Pavel Ustinov, Harish Srinivas, Kaikai Wang, Anthony Shoumikhin, Jesik Min, and Mani Malek. 2022. PAPAYA: Practical, Private, and Scalable Federated Learning. In *Proceedings of the 5th Conference on Machine Learning and Systems (MLSys 2022)*. https://proceedings.mlsys.org/paper_files/paper/2022/hash/a8bc4cb14a20f20d1f96188bd61ec87-Abstract.html
- [10] David Irwin, Prashant J. Shenoy, Pradeep Ambati, Prateek Sharma, Supreeth Shastri, and Ahmed Ali-Eldin. 2019. The Price Is (Not) Right: Reflections on Pricing for Transient Cloud Servers. In *28th International Conference on Computer Communication and Networks (ICCCN)*. 1–9.
- [11] K. R. Jayaram, Vinod Muthusamy, Gegi Thomas, Ashish Verma, and Mark Purcell. 2022. Adaptive Aggregation for Federated Learning. *arXiv preprint arXiv:2203.12163* (2022).
- [12] Pedro Joaquim, Manuel Bravo, Luis Rodrigues, and Miguel Matos. 2019. Hourglass: Leveraging Transient Resources for Time-Constrained Graph Processing in the Cloud. In *The Fourteenth EuroSys Conference (EuroSys)*.
- [13] E. L. Kaplan and Paul Meier. 1958. Nonparametric Estimation from Incomplete Observations. *J. Amer. Statist. Assoc.* 53, 282 (1958), 457–481. doi:10.1080/01621459.1958.10501452
- [14] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Collieran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. 2020. Lessons Learned from the Chameleon Testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association.
- [15] Alex Krizhevsky. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report. University of Toronto.
- [16] Sungjae Lee, Jaeh Hwang, and Kyungyong Lee. 2022. SpotLake: Diverse Spot Instance Dataset Archive Service. In *2022 IEEE International Symposium on Workload Characterization (IISWC)*. 242–255. doi:10.1109/IISWC55918.2022.00029
- [17] Mu Li, David G. Andersen, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Yiing Su. 2014. Scaling Distributed Machine Learning with the Parameter Server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 583–598.
- [18] Zhifei Li, Tian Xia, Ziming Mao, Zihan Zhou, Ethan J. Jackson, Jamison Kerney, Zhanghao Wu, Pratik Mishra, Yi Xu, Yifan Qiao, Scott Shenker, and Ion Stoica. 2026. SkyNomad: On Using Multi-Region Spot Instances to Minimize AI Batch Job Cost. *CoRR* abs/2601.06520 (2026). arXiv:2601.06520 doi:10.48550/arXiv:2601.06520
- [19] Yang Liu, Tao Fan, Tianjian Chen, Qian Xu, and Qiang Yang. 2021. FATE: An Industrial Grade Platform for Collaborative Learning with Data Protection. *Journal of Machine Learning Research* 22, 226 (2021), 1–6.
- [20] Heiko Ludwig, Nathalie Baracaldo, Gegi Thomas, Yi Zhou, Ali Anwar, Shashank Rajamoni, Yuya Ong, Jayaram Radhakrishnan, Ashish Verma, Mathieu Sinn, et al. 2020. IBM Federated Learning: An Enterprise Framework White Paper V0.1. *arXiv preprint arXiv:2007.10987* (2020).
- [21] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 1273–1282.
- [22] John Nguyen, Kshitiz Malik, Hongyuan Zhan, Ashkan Yousefpour, Mike Rabbat, Mani Malek, and Dzmitry Huba. 2022. Federated Learning with Buffered Asynchronous Aggregation. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 3581–3607.
- [23] NVIDIA. 2025. Efficient Federated Learning in the Era of LLMs with Message Quantization and Streaming. NVIDIA Technical Blog. <https://developer.nvidia.com/blog/efficient-federated-learning-in-the-era-of-llms-with-message-quantization-and-streaming/>
- [24] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*. USENIX Association, Philadelphia, PA, 305–319. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [25] Ljubica Pajevic, Gunnar Karlsson, and Viktoria Fodor. 2022. CRAWDAD kth/campus. doi:10.15783/c7-5r6x-4b46
- [26] Shixiong Qi, K. K. Ramakrishnan, and Myungjin Lee. 2024. LiFL: A Lightweight, Event-driven Serverless Platform for Federated Learning. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [27] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and H. Brendan McMahan. 2021. Adaptive Federated Optimization. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2003.00295>
- [28] Ruitao Shang, Fei Xu, Zhuoyan Bai, Li Chen, Zhi Zhou, and Fangming Liu. 2023. spotDNN: Provisioning Spot Instances for Predictable Distributed DNN Training in the Cloud. In *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*. 1–10.
- [29] Prateek Sharma, JCS Kadupitiya, and Vikram Jadhao. 2019. Modeling Constrained Preemption Dynamics Of Transient Cloud Servers. *arXiv* (2019), arXiv–1911.
- [30] Supreeth Subramanya, Amr Rizk, and David Irwin. 2016. Cloud Spot Markets are Not Sustainable: The Case for Transient Guarantees. In *8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*.
- [31] Yan Sun, Li Shen, Hao Sun, Liang Ding, and Dacheng Tao. 2023. Efficient Federated Learning Via Local Adaptive Amended Optimizer With Linear Speedup. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 12 (2023), 14453–14464. doi:10.1109/TPAMI.2023.3300886
- [32] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. 2023. Bamboo: Making Preemptible Instances Resilient for Affordable Training of Large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 497–513.
- [33] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H. Vincent Poor. 2020. Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2007.07481>
- [34] Zhanghao Wu et al. 2024. SkyServe: Serving AI Models across Regions and Clouds with Spot Instances. *arXiv preprint arXiv:2411.01438* (2024).
- [35] Zhanghao Wu, Wei-Lin Chiang, Ziming Mao, Zongheng Yang, Eric Friedman, Scott Shenker, and Ion Stoica. 2024. Can't Be Late: Optimizing Spot Instance Savings under Deadlines. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 185–203.
- [36] Cong Xie, Sanmi Koyejo, and Indranil Gupta. 2019. Asynchronous Federated Optimization. *arXiv preprint arXiv:1903.03934* (2019).
- [37] Fangkai Yang, Lu Wang, Zhenyu Xu, Jue Zhang, Liquan Li, Bo Qiao, Camille Couturier, Chetan Bansal, Soumya Ram, Si Qin, Zhen Ma, Íñigo Goiri, Eli Cortez, Terry Yang, Victor Rühle, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. Snape: Reliable and Low-Cost Computing with Mixture of Spot and On-Demand VMs. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.